

Presenting SecureGate to judges

This page is your script for showing SecureGate live: what to prepare, what to type, what the judges will see and what to say. The full demo takes about 12 minutes. Everything except the GitHub part works without the internet.

Type each command into **PowerShell**, in the project folder (see "Where do I type the commands?" below). Copy one block at a time. The quoted boxes are what to say: put them in your own words.

The demo at a glance

Step	What the judges see	What you type	Time
1	The problem SecureGate solves	nothing	1 min
2	A fake company's project, with secrets planted in it	<code>make demo</code>	1 min
3	The scan: masked values, block, warn and ignore; then four scanners and the pull request comment	<code>make scan-demo</code> , then <code>make scan-demo-all</code>	3 min
4	The rules, in one readable file	<code>notepad policy.yaml</code>	1 min
5	The dashboard in the browser	<code>make ui</code>	2 min
6	A commit stopped on the laptop	four commands	1 to 2 min
7	A pull request stopped on GitHub, with SecureGate's comment	nothing: a page you prepared	2 min
8	The proof: automatic tests and an honest scorecard	<code>make test</code>	1 min

Short on time? Run `make demo` before you start, then show steps 1, 3, 5 and 6 (about 7 minutes).

Prefer one window, and no commands to type? SecureGate's menu runs steps 2 to 5 for you: see "Present it from the menu" below.

Where does it start?

The one-click way: double-click `Start SecureGate.cmd` in the project folder. SecureGate opens in its own window: a padlock, its name in big letters, what is ready, and a menu. Type a number and press Enter. **1** builds the demo project (the first time only) and scans it with Gitleaks, like `make demo` and `make scan-demo`. **2** scans it with all four scanners, like `make scan-demo-all`, and then shows the comment a pull request would get on GitHub. After a scan, press Enter to open the dashboard in your browser, like `make ui`, and press Enter in the window again to close the dashboard. **3** shows the pull request comment again, **4** scans another project, **5** opens the dashboard on the last scan, **6** shows the rules, **7** builds the demo project again from scratch, **8** checks the setup, and **Q** closes the window. If SecureGate is not set up yet, it offers to do that first.

Under the hood, SecureGate is a **command-line program**: you type a command, it does its job, prints the result and stops. Only the dashboard keeps running, and you look at it in your web browser. The menu just types the commands for you: it shows each one, such as `> securegate scan ..\securegate-demo --mode repo --out findings-demo.json`, before running it.

- **The program** is `securegate`. `make setup` installed it inside the project folder, as `.venv\Scripts\securegate.exe`. You start it by typing a command, or let the menu type them.

- **The shortcuts.** Each `make` command is a short name for a longer command. They are all written in the file `Makefile` in the project folder. `make` prints the real command before running it, so the judges can see it.
- **The code.** Every command starts in the same place: the function `main()` in `src/securegate/cli.py`. The line `securegate = "securegate.cli:main"` in `pyproject.toml` connects the command's name to that function. From there, a scan runs Gitleaks (`scanners/gitleaks.py`) and, with `--scanners all`, TruffleHog, Semgrep and Bandit (the other files in `scanners/`), masks each value (`pipeline.py` and `mask.py`), lets the policy decide (`policy.py`), merges what several scanners found on one line (`merge.py`), then prints and saves the report (`report.py`) and, when asked, the pull request comment and SARIF (`outputs/`). How it's built ([how-its-built.pdf](#)) has the full map.

You type	Same as typing	What it does
<code>make setup</code>	<code>py -3.12 -m venv .venv</code> , then two <code>pip install</code> commands	installs SecureGate and its tools into <code>.venv</code> (once; needs the internet)
<code>make demo</code>	<code>.venv\Scripts\securegate demo-repo --out ..\securegate-demo --seed 42 --force</code>	builds the demo project
<code>make scan-demo</code>	<code>.venv\Scripts\securegate scan ..\securegate-demo --mode repo --out findings-demo.json</code>	scans the demo project's whole history
<code>make scan-demo-all</code>	the same, plus <code>--scanners all --no-verification --comment reports\demo-comment.md</code> and the summary and SARIF	scans with all four scanners and writes the comment a pull request would get into <code>reports\</code> (live checks off: the fake keys are never sent anywhere)
<code>make ui</code>	<code>.venv\Scripts\securegate ui --report findings-demo.json --open</code>	starts the dashboard and opens it in the browser
<code>make menu</code>	<code>.venv\Scripts\securegate menu</code>	opens the menu, like double-clicking <code>Start SecureGate.cmd</code>
<code>make test</code>	<code>.venv\Scripts\python.exe -m pytest</code>	runs the automatic tests and prints the scorecard
<code>make check</code>	the code style check (ruff), then the tests	the full check before every commit
<code>make hooks</code>	<code>.venv\Scripts\python.exe -m pre_commit install --install-hooks</code>	turns on the laptop gate (once; needs the internet)
<code>make scanners</code>	<code>.venv\Scripts\python.exe tools\install_scanners.py</code>	installs TruffleHog, Semgrep and Bandit into <code>.venv</code> at the pinned versions (once; needs the internet)
<code>make docs-pdf</code>	<code>.venv\Scripts\python.exe tools\docs_pdf.py</code>	rebuilds the PDFs in <code>docs\pdf</code>

If `make` ever stops working, type the command from the middle column instead: it does exactly the same.

Present it from the menu

Rather not type commands in front of the judges? Steps 2 to 5 can all be run from SecureGate's menu, in one window. Double-click `Start SecureGate.cmd`, maximize the window and make the text bigger (hold Ctrl and turn the mouse wheel). Each choice shows the command it runs, on a line starting with `>`, so the judges still see the real command, and the results are exactly those of the `make` commands in the steps below: their "Point at" and "Say" tips still apply.

Step	Type	What the judges see
2	7 , then y	The demo project is built again, exactly as new, as with <code>make demo</code> . Skip this if it is already built: 1 and 2 build it by themselves the first time.
3	1	The Gitleaks scan: the same table and Blocked: list as <code>make scan-demo</code> . The menu then waits at Open the results in the dashboard? . Talk about the table first, then type n to go back to the menu.
3	2	The scan with all four scanners, as with <code>make scan-demo-all</code> . The menu waits at Show the comment a pull request would get? : talk about the table and the Scanners: line first, then press Enter. The comment appears in the window; scroll up to its first line, SecureGate: BLOCKED (exit code 1) , and walk through it. Then type n at Open the results in the dashboard? .
3	3	The pull request comment again, at any time.
4	6	The rules in <code>policy.yaml</code> , top to bottom, each with its number, decision and reason.
5	5	The dashboard, on the last scan. Press Enter in the menu's window to close it and go back to the menu.

A few things are not in the menu: `git log` in step 2, changing `policy.yaml` in step 4 (choice 6 shows the rules; it does not change them), the commit in step 6, the GitHub page in step 7 and `make test` in step 8. Open a second PowerShell window in the project folder for them before you start.

Right before you start, choose **8**: it shows the versions of SecureGate and its four scanners, the rules, whether the laptop gate is on and whether the demo project is built. On the first screen, the **Scanners** line says `all four ready` when choice 2 can run. If it says `Gitleaks only`, run `make scanners` once (it needs the internet), or show choice 1 only.

Where do I type the commands?

In **PowerShell**, opened in the **project folder**: the folder with `Makefile`, `policy.yaml` and `src` in it (on this laptop, `Documents\Nexora`). The easiest way to get there:

1. Open the project folder in File Explorer.
2. Click the address bar at the top, type `powershell` and press Enter.

PowerShell opens, already in the right folder. Another way: open PowerShell from the Start menu and type `cd $HOME\Documents\Nexora`. Or, in VS Code, choose **File > Open Folder**, pick the project folder, then **Terminal > New Terminal**.

To check that you are in the right folder:

```
Test-Path Makefile
```

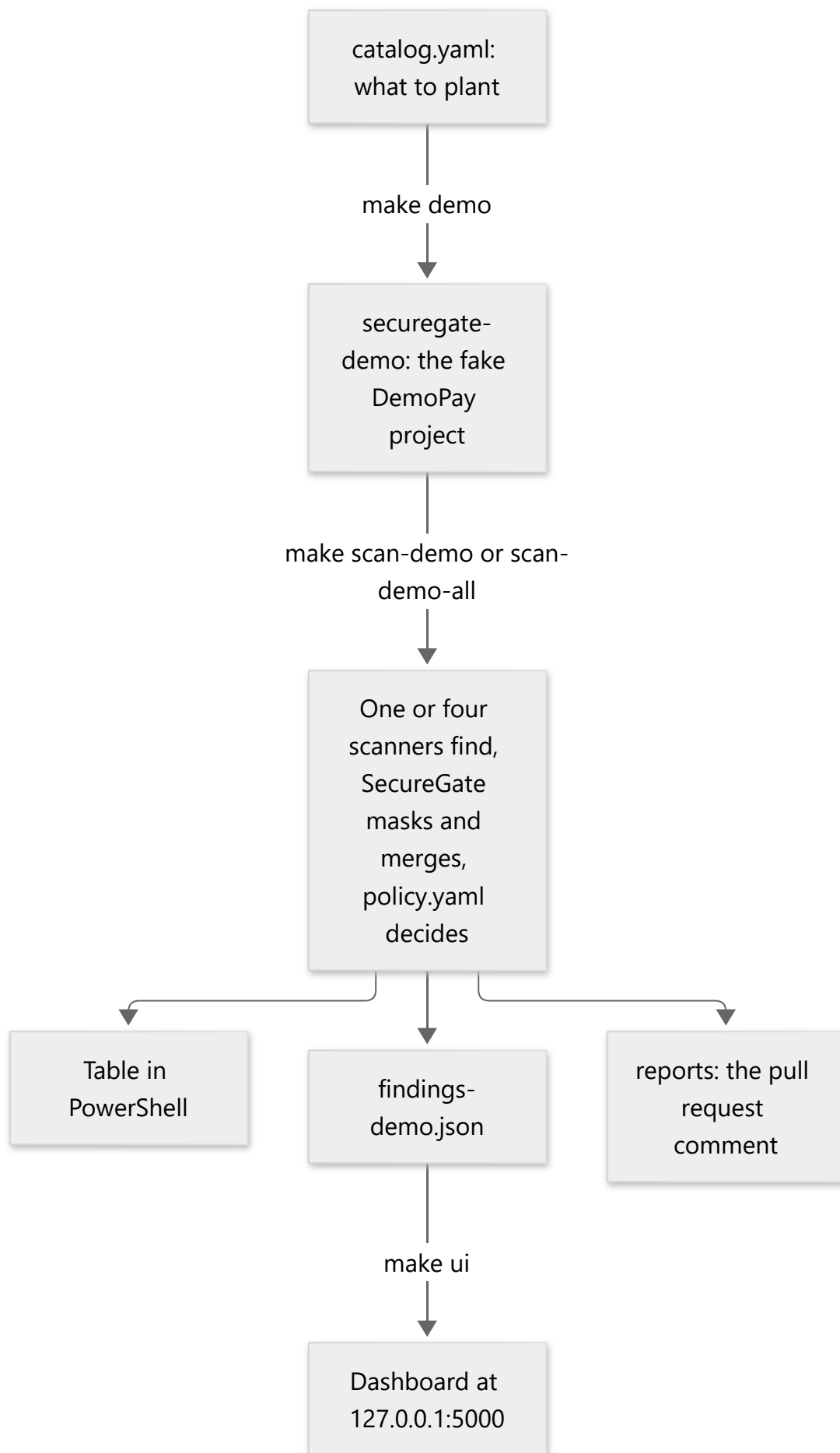
It prints `True`. If it prints `False`, you are in another folder.

Make the text big enough for the judges: hold Ctrl and turn the mouse wheel.

The dashboard is the only part you see in a web browser, at `http://127.0.0.1:5000`. That address only works on this laptop, and only while `make ui` runs or the menu shows the dashboard.

Where does the data come from?

Everything the judges see is made on the spot, on this laptop, from files in the project. No real secret, no database and no internet connection is involved.



1. **The plan:** `src/securegate/demo/catalog.yaml`. A list of 20 lines to plant: 9 fake secrets (AWS, Stripe, GitHub and ACME Pay keys, a private key and two passwords) and 11 **decoys**, values that only look like secrets (placeholders, a commit hash, an image written as text, and more).
2. **The demo project:** `make demo`. It builds DemoPay, a small fake payments app, in a folder called `securegate-demo` next to the project folder. DemoPay is a real Git project with 8 commits by an invented developer, Riya Demo. Each fake key is made at that moment: the right beginning, such as `sk_live_` for a Stripe key, plus random characters. The random choices come from seed 42, a fixed number, so every run builds exactly the same project and gives the same results.
3. **The hidden key.** In the third commit, Riya adds a script with a Stripe key in it. In the seventh, she deletes the script. The key is gone from the latest version of the code, but it is still in the history.
4. **The answer sheet:** `ground_truth.csv`, in the `securegate-demo` folder. It lists every planted line and the decision SecureGate should make, never the values. The tests in step 8 build their own copy of the demo and compare SecureGate's results with its answer sheet: that is the scorecard.
5. **The scan:** `make scan-demo`. SecureGate starts **Gitleaks**, a free scanner, which reads all 8 commits and reports every piece of text shaped like a key. Gitleaks hands its results over in a temporary folder, which SecureGate reads and deletes at once. SecureGate masks each value, makes its fingerprint and asks `policy.yaml` for a decision. It prints the table and saves the report as `findings-demo.json` in the project folder. It never prints or saves a whole value. `make scan-demo-all` does the same with all four scanners: TruffleHog reads the 8 commits too, and Semgrep and Bandit read today's files. It also writes the comment that a pull request would get into `reports/demo-comment.md`. TruffleHog's live checks are switched off there, so the fake keys are never sent to Stripe, AWS or GitHub.
6. **The dashboard:** `make ui`. It reads `findings-demo.json`, and nothing else, every time a page opens.

Three more things come up in the demo:

- `securegate demo-token` makes a new, random fake token for ACME Pay, a payment provider invented for SecureGate's demos. Steps 6 and 7 use it.
- `sample_findings.json` is a ready-made report with 22 findings, kept in the project as a spare. If a scan fails on the day, you can still show the dashboard with it.
- `.securegate\local.key` is a private random key that the first scan creates. SecureGate only uses it to make fingerprints. It stays on this laptop (Git ignores it). Never share it.

The day before

Do this with the internet on. Then rehearse the whole demo once, with a timer.

1. Open PowerShell in the project folder and check the tools:

```
.venv\Scripts\securegate version
```

It prints `securegate 0.1.0` and `gitleaks 8.30.1`, then a line for each of the other three scanners: `trufflehog 3.97.9`, `semgrep 1.179.0` and `bandit 1.9.4` once step 2 has installed them, `not found` before. If it says `gitleaks not found`, or PowerShell does not recognize `.venv\Scripts\securegate`, follow steps 1 and 3 of Getting started ([getting-started.pdf](#)).

2. Install the other three scanners, for step 3 (once; needs the internet):

```
make scanners
```

It ends with Semgrep 1.179.0 and Bandit 1.9.4: installed. Then try `make scan-demo-all` once: it ends with 12 findings: 6 block, 5 warn, 1 ignore.

3. Run every check:

```
make check
```

It takes about a minute and ends with a line like 746 passed, 1 skipped. The skipped test only runs on macOS and Linux. If any test failed, fix that before the day.

4. Turn on the laptop gate, for step 6:

```
make hooks
```

It prints `pre-commit` installed at `.git\hooks\pre-commit`.

5. For step 7, prepare the GitHub part (next section). Skip it if you won't show GitHub.
6. Save a backup: screenshots of the dashboard, of `reports\demo-comment.md` and of the GitHub pull request with SecureGate's comment, in case something fails on the day.
7. If you can, try the projector. On wide screens such as 1920 × 1080, the dashboard uses bigger text by itself.

Prepare the GitHub part (once)

You need a GitHub account and about 15 minutes.

1. **Check that the project is on GitHub.** SecureGate's repository is `github.com/Bhavik-Kadian/SecureGate`, and its `main` already runs the four-scanner gate. In the project folder, `git remote -v` lists it as `origin`.
2. **Open the demo pull request:** follow steps 1 to 6 of Demo: the merge gate (`demo-merge-gate.pdf`). Stop after step 6, and keep its step 7 (clean up) for after the judging, so the pull request is still there to show. The check turns red a minute or two after each push.
3. **Make the check required:** follow "The one-time setting" in The two gates (`merge-gate.pdf`). The pull request then says that merging is blocked. From now on, changes only reach `main` through a pull request whose check passed: that is the point.
4. **Before you present,** open the pull request in a browser tab, signed in to GitHub.

Right before you start (5 minutes)

1. Turn on **Do not disturb** in Windows, and close the apps you don't need.
2. Open PowerShell in the project folder and make the text bigger.
3. Check the scanners: `.venv\Scripts\securegate version` prints `gitleaks 8.30.1`, `trufflehog 3.97.9`, `semgrep 1.179.0` and `bandit 1.9.4`.

4. Check that the laptop gate is on: `Test-Path .git\hooks\pre-commit` prints `True`. If it prints `False`, run `make hooks` (needs the internet) or skip step 6. Or check steps 3 and 4 in one go: double-click `Start SecureGate.cmd` and choose 8. If you present from the menu, keep that window open.
5. For step 7, open the pull request in a browser tab.
6. Type `cls` to clear the screen.

The demo, step by step

Step 1. The problem (1 minute)

Nothing to type.

Developers sometimes paste passwords and keys into their code by mistake. Once that is committed, deleting the line does not help: Git keeps every old version, and anyone with a copy can read the key. Attackers run bots that search public code for keys all day. SecureGate stops secrets before they get in. Four scanners look for them: Gitleaks and TruffleHog search every commit, and Semgrep and Bandit check the code itself. One readable file, `policy.yaml`, decides what to do with what they find: block, warn or ignore. And SecureGate never shows a whole secret, not even in its own reports.

Step 2. Build a fake company's project (1 minute)

```
make demo
```

The judges see:

```
".venv/Scripts/python.exe" -m securegate demo-repo --out "../securegate-demo" --seed 42 --force
Demo repo ready: C:\Users\...\Documents\securegate-demo
  8 commits by Riya Demo, seed 42
  planted: 9 secret lines and 11 decoy lines
  ground truth: C:\Users\...\Documents\securegate-demo\ground_truth.csv
Next: scan it with `make scan-demo`
```

This builds DemoPay, a fake payments app, as a real Git project with 8 commits. We planted 9 fake secrets in it, and 11 decoys: things that only look like secrets. All of them are made up on the spot and work nowhere.

Then show its history:

```
git -C ../securegate-demo log --oneline
```

```
52f88ba Add tests and docs
0c21e59 Remove one-off maintenance scripts
6ecda21 Add Docker build
3afaea1 Add checkout page
d7e7354 Add Stripe payments client
720d024 Add one-off maintenance scripts
f6b8246 Add configuration
fcfaa5c Start the DemoPay payments service
```

The newest commit is at the top. In "Add one-off maintenance scripts", Riya pasted a Stripe key into a script. Four commits later, she deleted the script. The key is gone from today's code. Let's see whether it is really gone.

Step 3. Scan it, with one scanner and then with four (3 minutes)

```
make scan-demo
```

The judges see (the "Blocked" part is shortened here):

```
".venv/Scripts/python.exe" -m securegate scan "../securegate-demo" --mode repo --out findings-demo.json
```

DECISION	RULE	FILE:LINE	VALUE
block	github-pat	.env:2	ghp_****pVJI
block	aws-access-token	config/settings.py:11	AKIA****NOBM
block	private-key	deploy/server.key:1	-----****-----
block	stripe-access-token	payments/stripe_client.py:5	sk_l****0SJo
block	stripe-access-token	scripts/migrate_customers.py:3	sk_l****562d
block	acme-pay-token	web/checkout.js:2	acme****DHqJ
warn	generic-api-key	config/app.yaml:8	d450****df49
warn	generic-api-key	config/settings.py:12	1C5J****6UuV
warn	stripe-access-token	tests/fixtures/stripe_webhook.json:3	sk_t****Fbyv
warn	generic-api-key	web/version.js:2	810a****d785
ignore	acme-pay-token	docs/payments.md:10	acme****xxxx

Blocked:

```
.env:2  ghp_****pVJI
  why: provider-keys: A payment, cloud or private key gives direct access to money, data
or servers.
  fix: Treat it as leaked. Rotate (replace) the key at the provider now, ...
config/settings.py:11  AKIA****NOBM
  why: provider-keys: A payment, cloud or private key gives direct access to money, data
or servers.
  fix: Treat it as leaked. Rotate (replace) the key at the provider now, ...
... (one entry for each of the 6 blocked findings)
```

```
11 findings: 6 block, 4 warn, 1 ignore -> BLOCKED (exit code 1). Details: findings-demo.json
make: *** [Makefile:47: scan-demo] Error 1
```

Point at	Say
any value	"Every value is masked: only the first 4 and last 4 characters are shown. SecureGate never prints, logs or saves a whole secret."
scripts/migrate_customers.py:3	"This is the Stripe key Riya deleted. It is not in today's code, but SecureGate reads every commit, so it still finds it."
the Blocked: part	"For every blocked secret, it says why and how to fix it. Both come straight from policy.yaml."
docs/payments.md:10	"This one is shaped like a real key, but it is all x's: a placeholder in the docs. The policy ignores it, so it bothers nobody."
the last two lines	"Exit code 1 means 'something is blocked'. That is the signal that stops a commit or a pull request. make shows it as 'Error 1', which is expected here."

If you have time, show why reading the history matters. This scans only the files as they are today:

```
.venv\Scripts\securegate scan ..\securegate-demo --mode dir --out findings-dir.json
```

It ends with 10 findings: 5 block, 4 warn, 1 ignore: the deleted Stripe key is missing.

A normal scan of today's files misses the deleted key. That is why SecureGate reads the whole history by default.

Now the same scan with all four scanners:

```
make scan-demo-all
```

The table has one more line, and the end says which scanners took part:

```
warn      trufflehog-postgres  config/app.yaml:7      post****5432
...
Scanners: Gitleaks 8.30.1; TruffleHog 3.97.9 (live checks switched off with --no-
verification); Semgrep 1.179.0; Bandit 1.9.4
12 findings: 6 block, 5 warn, 1 ignore -> BLOCKED (exit code 1). Details: findings-demo.json
```

Point at	Say
config/app.yaml:7	"Gitleaks missed this one: a password inside a database address. TruffleHog found it. Each scanner catches things the others miss."
the Scanners: line	"Four scanners, one verdict. When several find the same key on the same line, SecureGate reports it once and names every scanner that saw it."
live checks switched off	"On GitHub, TruffleHog also asks the provider whether a key still works. Here that is switched off, because these fake keys must never be sent to Stripe or AWS."

Then show what a pull request would get. Open `reports\demo-comment.md` in VS Code and press **Ctrl+Shift+V** for the preview (or `notepad reports\demo-comment.md` for the plain text):

This is the comment SecureGate posts on a pull request. The first line is the verdict. The table says which rule decided, where, the masked value, which scanners found it, and whether the key is live. For every blocked key there is a checklist to rotate it, and it ends with: deleting the line is not enough, the key stays in Git history. Warnings are explained too, so nobody has to guess.

Step 4. The rules, in one readable file (1 minute)

```
notepad policy.yaml
```

This one file decides. The rules are checked from top to bottom, and the first one that matches wins. Each rule has its number from our policy table. A key that the provider confirms is live is always blocked. Placeholders are ignored. Anything in tests, fixtures, docs or Markdown only gets a warning. Live payment, cloud and GitHub keys and private keys are blocked. Test-mode keys, passwords written in the code and risky handling of secrets get a warning, and so does anything else. A security team can read and change this without touching the code.

Close Notepad without saving.

If a judge asks you to change a rule, you can do it live:

1. In Notepad, find `name: hardcoded-passwords` (rule 10). Change the `decision: warn` below it to `decision: block`, and save with Ctrl+S.
2. Run `make scan-demo` again. The last line now says `11 findings: 9 block, 1 warn, 1 ignore`: the three findings that this rule covers, values written next to a word like key or password, are now blocked.
3. Undo the change, and scan again with all four scanners, so that the dashboard shows the normal result (use `make scan-demo` if the other scanners are not installed):

```
git restore policy.yaml
make scan-demo-all
```

SecureGate never guesses when the file has a mistake, such as `decision: stop`. It says what is wrong, for example rule 10 ('hardcoded-passwords'): 'decision' must be one of: block, warn, ignore, and ends with exit code 2. On an error, it never lets anything through.

Step 5. The dashboard (2 minutes)

```
make ui
```

PowerShell shows:

```
SecureGate dashboard: http://127.0.0.1:5000/
Showing findings-demo.json. Press Ctrl+C to stop.
```

and the dashboard opens in your browser. PowerShell stays busy while the dashboard runs, and adds a line for each page the browser loads. That is normal.

Show, in this order:

1. **Overview.** The red bar at the top: "BLOCKED: 6 findings are blocked", and "exit code 1". The tiles: 12 findings, 6 blocked, 5 warnings, 1 ignored (11 and 4 after a scan with Gitleaks alone). **Fix these first** lists the 6 blocked findings, the most severe first. The bars count the findings by severity. "About this scan" says what was scanned: the whole Git history, with Gitleaks 8.30.1 and `policy.yaml`.
2. Select the **Blocked** tile. The list now shows only the 6 blocked findings.
3. Select the row `scripts/migrate_customers.py:3`, the deleted Stripe key. Its page shows `sk_l****562d` with a BLOCK badge, **How to fix** in four numbered steps (starting with "Revoke it at the provider") next to the facts at a glance, and then every field: the commit that added the key, its author Riya Demo, the date, the fingerprint, the confidence score, which scanners found it (**Found by:** gitleaks, trufflehog), the **Live check** and the **Policy rule** (`rule 8: provider-keys`).
4. If there is time, select **Report** at the top: the whole report on one page. Press Ctrl+P to show that it prints, light, or saves as a PDF. The **CSV** button downloads the findings for Excel.

The dashboard is read-only and runs only on this laptop: 127.0.0.1 means "this computer", so nobody else on the network can open it. It needs no internet, and its pages contain no JavaScript. It checks every value again before showing it: a report that holds an unmasked value is refused, never shown, and never downloaded. The downloads and the printed report only ever hold masked values too.

To stop the dashboard, click in PowerShell and press **Ctrl+C**. Tip: to keep it open for the rest of the demo, run `make ui` in a second PowerShell window instead. In SecureGate's menu, 5 opens the dashboard, and Enter in the menu's window closes it.

Step 6. The laptop gate stops a commit (1 to 2 minutes)

Only do this if `Test-Path .git\hooks\pre-commit` printed `True` (see "Right before you start").

Now I am a developer about to commit a key by mistake. This is a fake token for ACME Pay, a payment provider we invented for demos. It is new and random every time, and it unlocks nothing.

```
$token = .venv\Scripts\securegate demo-token
Set-Content -Path demo_leak.py -Value "ACME_PAY_API_KEY = '$token'" -Encoding utf8
git add demo_leak.py
git commit -m "Demo: add a fake ACME Pay token"
```

The first line makes a token and keeps it without showing it. The second writes it into a one-line file. The last two try to commit that file. The judges see:

```
SecureGate secret scan (staged changes).....Failed
- hook id: securegate
- exit code: 1

DECISION  RULE          FILE:LINE    VALUE
block     acme-pay-token  demo_leak.py:1  acme****4HKt

Blocked:
  demo_leak.py:1  acme****4HKt
    why: provider-keys: A payment, cloud or private key gives direct access to money, data
or servers.
    fix: Treat it as leaked. Rotate (replace) the key at the provider now, remove it from
the code, and load it from an environment variable or a secrets manager instead.

1 finding: 1 block, 0 warn, 0 ignore -> BLOCKED (exit code 1). Details: .securegate\staged-
findings.json
```

The masked value is different every time, because the token is new. A warning about CRLF and LF after `git add` is harmless.

The commit did not happen. Before the key ever leaves the laptop, the developer sees the file and line, the masked key, why it was blocked and how to fix it. It only checks the changes being committed, so it is quick and works offline.

Always clean up afterwards:

```
git restore --staged demo_leak.py
Remove-Item demo_leak.py
git status --short
```

`git status --short` no longer lists `demo_leak.py`.

If the commit went through (the laptop gate was not on), first undo it with `git reset --soft HEAD~1`, then run the three clean-up commands. Never push a commit that holds a demo token to `main`.

A developer can skip this check with `git commit --no-verify`. That is allowed on purpose: a laptop check is an early warning, not a control. The real control is the second gate, on GitHub.

Step 7. The merge gate stops a pull request (1 to 2 minutes)

This needs the internet and the GitHub preparation. In the pull request's browser tab, show:

1. The check **secret-gate** with a red cross, and the box at the bottom saying that merging is blocked.
2. **SecureGate's comment** on the pull request, posted by `github-actions`: "SecureGate: BLOCKED (exit code 1)", `demo_leak.py:1` with the masked token, **rule 8: provider-keys**, found by Gitleaks and TruffleHog, and a checklist to rotate an ACME Pay key that ends with "Deleting the line is not enough: the key stays in Git history." Each push updates this one comment instead of adding another.
3. The pull request's **Commits** tab: the second commit deleted the token, yet the check is still red.
4. If there is time: select **Details** next to the check, then **Summary** at the top left, for the same report on the check's page; and the repository's **Security** tab, where the blocked token appears as a code scanning alert.

This is the real control. It runs on GitHub's computers, for every pull request, whoever opened it, with all four scanners. It scans every commit in the pull request, so deleting the key later does not help: it is still in the history, where anyone can read it. It judges each pull request with the rules from the main branch, so a pull request cannot weaken the rules that judge it. It explains itself in the pull request, with a checklist to rotate the key. And with this setting, nobody, not even an admin, can merge while the check is red.

If the judges want to see how it works, open `.github/workflows/secret-gate.yml`, in VS Code or on GitHub. Every step has a comment in plain English, the four tools are pinned to exact versions in one place at the top, and Gitleaks and TruffleHog are checked against their published checksums before they run. A good proof: pull request #2 on SecureGate added this four-scanner workflow, and was the first pull request it judged.

No internet? Show your screenshots, `reports\demo-comment.md` from step 3, or `docs/pdf/merge-gate.pdf`, and make the same points.

Step 8. The proof: tests and an honest scorecard (1 minute)

```
make test
```

After about a minute, the judges see the scorecards, and then the last line: `746 passed, 1 skipped`.

```
repo mode, seed 42
planted lines: 20 (9 secret, 11 decoy)
  hits          15
  misses        2
    db_url_password at config/app.yaml:7: expected warn, got nothing
    generic_password at Dockerfile:6: expected warn, got nothing
wrong decisions  1
  aws_key_pair at config/settings.py:12: expected block, got warn
false alarms    2
  uuid at config/app.yaml:8: expected ignore, got warn
  git_sha at web/version.js:2: expected ignore, got warn
```

A second scorecard follows for `dir` mode. It only reads today's files, so it also misses the deleted Stripe key. With the other three scanners installed, more follow: one for all four scanners together (16 hits and only 1 miss) and one for each scanner alone, which shows what each of them adds.

Over 700 automatic tests check SecureGate on every change. Some run the real Gitleaks, TruffleHog, Semgrep and Bandit; others fail if a whole fake secret ever appears in any output: a report, a dashboard page, a pull request comment or a SARIF file. The scorecard compares a fresh scan of the demo with its answer sheet, and we show the real numbers: Gitleaks alone handles 15 of the 20 planted lines exactly as expected; all four scanners together handle 16, and only one password is still missed, inside a Dockerfile. That is our next improvement.

Word	Meaning
hit	decided as expected; for a decoy, ignored or not reported at all
miss	a secret that was not reported: the most dangerous mistake
wrong decision	found, but warned about instead of blocked, or the other way round
false alarm	a decoy reported as block or warn

The close

To sum up: four scanners find, one readable policy decides, and nobody ever sees a whole secret. Two gates stop leaks: one on the laptop, and the real one on GitHub, which explains itself in every pull request.

After the demo

- Stop the dashboard if it is still running: press Ctrl+C in its PowerShell window, or Enter in the menu's window. Choose Q to close the menu.
- Check that `demo_leak.py` is gone: `git status --short` does not list it.
- On GitHub, finish the demo pull request: step 7 of Demo: the merge gate (demo-merge-gate.pdf) closes it without merging and deletes the branch.
- You can run `make demo` again at any time. It rebuilds exactly the same demo project.

Questions judges may ask

Is any real secret used? No. Every value in the demo is made up when the demo is built, and ACME Pay does not exist. The project's own files hold no key-shaped text at all: a test scans the project itself to make sure.

Where are the secrets stored? Nowhere. A report holds only a masked value and a **fingerprint**: a code that recognizes the same secret again, but cannot be turned back into it. It is made with HMAC-SHA256 and the private key in `.securegate`.

Why four scanners, and not just Gitleaks? Each catches things the others miss: TruffleHog found the password inside a database address that Gitleaks missed, and can ask the provider whether a key still works; Semgrep and Bandit read code, and catch a secret printed to a log or a password typed into Python. SecureGate merges what they find into one verdict and adds what a team needs to act on it: one readable policy that decides block, warn or ignore; masking everywhere; a reason and a fix for every finding; one clear exit code for the gates; the dashboard; and the two gates themselves.

What if a developer skips the laptop check? `git commit --no-verify` skips it, on purpose. The merge gate on GitHub still scans every commit in the pull request, and the ruleset stops the merge.

Can a pull request change the rules to let its own secret through? No. The merge gate takes SecureGate and all its rules (`policy.yaml`, `.gitleaks.toml`, `.trufflehog.yaml` and `rules/`) from the main branch, not from the pull request. The one exception is a pull request that changes the workflow file itself, so the ruleset should also require a review. See The two gates ([merge-gate.pdf](#)).

What if a scanner is missing or crashes, or the policy has a mistake? Gitleaks and TruffleHog are required: if either fails, or the policy has a mistake, SecureGate stops with exit code 2, says why, and the check turns red. It never says "pass" when it could not do its job: it **fails closed**. Semgrep and Bandit are extra eyes: if one fails, the scan goes on, and the comment says in red that it did not run.

Doesn't deleting the key fix it? No. The key stays in the Git history, where anyone with a copy can read it. The real fix is **rotation**: make a new key at the provider and cancel the old one. That is why the fix for every blocked key starts with rotating or revoking it.

What about false alarms? Placeholders and documentation examples are ignored. Findings in tests, fixtures and docs only get a warning. Anything the policy is not sure about is a warning, not a block, so the gates only stop work for key types that are clearly dangerous. The scorecard shows the real numbers.

Can it tell whether a key still works? On GitHub, yes, for the providers TruffleHog knows: it asks the provider, and a key that still works is always blocked as critical (rule 1). In the demos that is switched off, so the fake keys are never sent anywhere. Either way, every blocked key is treated as leaked, which is the safe choice.

Can it scan any project? Yes, any Git project: `.venv\Scripts\securegate scan C:\path\to\project --mode repo`. See step 5 of Getting started ([getting-started.pdf](#)).

Does it need the internet? No. Scans, the dashboard and the laptop gate work offline. Only the merge gate needs GitHub. Semgrep downloads its p/secrets rules; without the internet it runs SecureGate's own rules and says so.

What is it built with? Python 3.12; Gitleaks 8.30.1, TruffleHog 3.97.9, Semgrep 1.179.0 and Bandit 1.9.4 for finding; Flask for the dashboard; pre-commit for the laptop gate; and GitHub Actions for the merge gate, with 747 automatic tests.

If something goes wrong

What you see	What to do
<code>make</code> : The term 'make' is not recognized	Close PowerShell and open it again. If it still happens, type the command from the middle column of the table in "Where does it start?".
<code>make scan-demo</code> ends with Error 1	Nothing: that is expected. It means secrets were found and blocked.
<code>make scan-demo-all</code> ends with Error 2 and TruffleHog was not found	Run <code>make scanners</code> once (needs the internet), or show <code>make scan-demo</code> instead.
The menu's choice 2 says it needs TruffleHog, Semgrep and Bandit	The same: run <code>make scanners</code> once, then open the menu again. Or show choice 1 instead.
The Scanners: line says Semgrep could not load p/secrets	There is no internet, so Semgrep ran SecureGate's own rules only. That is expected offline; the scan still counts.
Error 2 , or a line starting with securegate: error:	SecureGate could not do its job, and the line says why. Often Gitleaks is not found: run <code>.venv\Scripts\securegate version</code> . If it says gitleaks not found , close PowerShell and open it again, or install Gitleaks again (step 1 of Getting started (getting-started.pdf)).
<code>.venv\Scripts\securegate</code> is not recognized	You are in the wrong folder (check with Test-Path Makefile), or SecureGate is not installed yet: run <code>make setup</code> (needs the internet).
cannot start the dashboard on port 5000	A dashboard is already open, maybe in another window, or another program is using that port. Use the open dashboard and reload its page, or run <code>.venv\Scripts\securegate ui --report findings-demo.json --port 5050 --open</code> .
The browser did not open	Type <code>http://127.0.0.1:5000</code> into the browser's address bar yourself.
The dashboard says there is no report	Run <code>make scan-demo</code> in a second PowerShell window, then reload the page.
PowerShell seems stuck after <code>make ui</code>	It is not stuck: it is running the dashboard. Ctrl+C stops it.
The commit in step 6 went through	The laptop gate was not on. Undo the commit with <code>git reset --soft HEAD~1</code> , then run the clean-up commands of step 6.
No internet where you present	Steps 1 to 6 and step 8 work offline. For step 7, show your screenshots and <code>reports\demo-comment.md</code> .
The window of Start SecureGate.cmd says SecureGate stopped	The lines above it say what went wrong, often one of the problems in this table. Press a key to close the window, fix the problem, then double-click the file again.
The window of Start SecureGate.cmd asks Terminate batch job (Y/N)?	Someone pressed Ctrl+C. Press Y to close the window, then double-click the file again. To close SecureGate normally, choose Q in its menu.
A scan fails on the day and you can't fix it	Show the spare report: <code>.venv\Scripts\securegate ui --report sample_findings.json --open</code> .

Where everything is

Where	What it is
Makefile	the shortcut commands (<code>make ...</code>)
Start SecureGate.cmd	double-click it to open SecureGate's menu: scan the demo (with one scanner or all four) or another project, show the pull request comment, open the dashboard, see the rules

Where	What it is
<code>.venv\</code>	SecureGate and its tools, installed by <code>make setup</code> . The program is <code>.venv\Scripts\securegate.exe</code> .
<code>src/securegate/</code>	the program's code. Every command starts in <code>cli.py</code> .
<code>policy.yaml</code>	the rules: block, warn or ignore
<code>.gitleaks.toml</code>	what Gitleaks looks for, including our rule for ACME Pay tokens
<code>.trufflehog.yaml</code>	TruffleHog's extra detector, for ACME Pay tokens
<code>rules\securegate-risky.yml</code>	SecureGate's own Semgrep rules: a secret printed to a log, put in a URL, or given a default in the code
<code>src/securegate/demo/catalog.yaml</code>	what the demo project plants
<code>..\securegate-demo\</code>	the demo project that <code>make demo</code> builds, next to the project folder
<code>findings-demo.json</code>	the report of the last <code>make scan-demo</code> or <code>make scan-demo-all</code> , with masked values only
<code>reports\</code>	the pull request comment, summary and SARIF of the last <code>make scan-demo-all</code> (Git ignores this folder)
<code>sample_findings.json</code>	a spare report with 22 findings
<code>.securegate\</code>	the private fingerprint key, and the laptop gate's last report. Never share it.
<code>.pre-commit-config.yaml</code> , <code>tools/precommit_hook.py</code>	the laptop gate
<code>.github/workflows/secret-gate.yml</code>	the merge gate
<code>tests/</code>	the automatic tests
<code>docs/</code> and <code>docs/pdf/</code>	these pages, and a PDF copy of each